the difference between the interfaces algebraically. But here are a few other common ways of stating the difference:

- `Applicative` computations have fixed structure and simply *sequence* effects, whereas monadic computations may choose structure dynamically, based on the result of previous effects.
- `Applicative` constructs *context-free* computations, while `Monad` allows for *context sensitivity*.[2]
- `Monad` makes effects first class; they may be generated at "interpretation" time, rather than chosen ahead of time by the program. We saw this in our `Parser` example, where we generated our `Parser[Row]` *as part of* the act of parsing, and used this `Parser[Row]` for subsequent parsing.

12.4 The advantages of applicative functors

The `Applicative` interface is important for a few reasons:

- In general, it's preferable to implement combinators like `traverse` using as few assumptions as possible. It's better to assume that a data type can provide `map2` than `flatMap`. Otherwise we'd have to write a new `traverse` every time we encountered a type that's `Applicative` but not a `Monad`! We'll look at examples of such types next.
- Because `Applicative` is "weaker" than `Monad`, this gives the *interpreter* of applicative effects more flexibility. To take just one example, consider parsing. If we describe a parser without resorting to `flatMap`, this implies that the structure of our grammar is determined before we begin parsing. Therefore, our interpreter or runner of parsers has *more information* about what it'll be doing up front and is free to make additional assumptions and possibly use a more efficient implementation strategy for running the parser, based on this known structure. Adding `flatMap` is powerful, but it means we're generating our parsers dynamically, so the interpreter may be more limited in what it can do. Power comes at a cost. See the chapter notes for more discussion of this issue.
- Applicative functors compose, whereas monads (in general) don't. We'll see how this works later.

12.4.1 Not all applicative functors are monads

Let's look at two examples of data types that are applicative functors but not monads. These are certainly not the only examples. If you do more functional programming, you'll undoubtedly discover or create lots of data types that are applicative but not monadic.[3]

---

[2] For example, a monadic parser allows for context-sensitive grammars while an applicative parser can only handle context-free grammars.

[3] *Monadic* is the adjective form of *monad*.

THE APPLICATIVE FOR STREAMS

The first example we'll look at is (possibly infinite) streams. We can define `map2` and `unit` for these streams, but not `flatMap`:

```
val streamApplicative = new Applicative[Stream] {

  def unit[A](a: => A): Stream[A] =
    Stream.continually(a)

  def map2[A,B,C](a: Stream[A], b: Stream[B])(
                  f: (A,B) => C): Stream[C] =
    a zip b map f.tupled
}
```

The infinite, constant stream.

Combine elements pointwise.

The idea behind this `Applicative` is to combine corresponding elements via zipping.

EXERCISE 12.4

Hard: What is the meaning of `streamApplicative.sequence`? Specializing the signature of `sequence` to `Stream`, we have this:

```
def sequence[A](a: List[Stream[A]]): Stream[List[A]]
```

VALIDATION: AN EITHER VARIANT THAT ACCUMULATES ERRORS

In chapter 4, we looked at the `Either` data type and considered the question of how such a data type would have to be modified to allow us to report multiple errors. For a concrete example, think of validating a web form submission. Only reporting the first error means the user would have to repeatedly submit the form and fix one error at a time.

This is the situation with `Either` if we use it monadically. First, let's actually write the monad for the partially applied `Either` type.

EXERCISE 12.5

Write a monad instance for `Either`.

```
def eitherMonad[E]: Monad[({type f[x] = Either[E, x]})#f]
```

Now consider what happens in a sequence of `flatMap` calls like the following, where each of the functions `validName`, `validBirthdate`, and `validPhone` has type `Either[String, T]` for a given type `T`:

```
validName(field1) flatMap (f1 =>
validBirthdate(field2) flatMap (f2 =>
validPhone(field3) map (f3 => WebForm(f1, f2, f3))
```

If `validName` fails with an error, then `validBirthdate` and `validPhone` won't even run. The computation with `flatMap` inherently establishes a linear chain of dependencies. The variable `f1` will never be bound to anything unless `validName` succeeds.

Now think of doing the same thing with `map3`:

```
map3(
  validName(field1),
  validBirthdate(field2),
  validPhone(field3))(
  WebForm(_,_,_))
```

Here, no dependency is implied between the three expressions passed to `map3`, and in principle we can imagine collecting any errors from each `Either` into a `List`. But if we use the `Either` monad, its implementation of `map3` in terms of `flatMap` will halt after the first error.

Let's invent a new data type, `Validation`, that is much like `Either` except that it can explicitly handle more than one error:

```
sealed trait Validation[+E, +A]

case class Failure[E](head: E, tail: Vector[E] = Vector())
  extends Validation[E, Nothing]

case class Success[A](a: A) extends Validation[Nothing, A]
```

EXERCISE 12.6

Write an `Applicative` instance for `Validation` that accumulates errors in `Failure`. Note that in the case of `Failure` there's always at least one error, stored in `head`. The rest of the errors accumulate in the `tail`.

To continue the example, consider a web form that requires a name, a birth date, and a phone number:

```
case class WebForm(name: String, birthdate: Date, phoneNumber: String)
```

This data will likely be collected from the user as strings, and we must make sure that the data meets a certain specification. If it doesn't, we must give a list of errors to the user indicating how to fix the problem. The specification might say that `name` can't be empty, that `birthdate` must be in the form `"yyyy-MM-dd"`, and that `phoneNumber` must contain exactly 10 digits.